IEORE4004: Optimization Models & Methods **Fall 2024**

Dynamic Programming

1 Introduction

Dynamic Programming is a technique to solve optimization problems that, roughly speaking, have a recursive structure. The basic idea is to solve a sequence of increasingly difficult suproblems of our original problem, with the following two properties:

1. Every problem in the sequence can be easily solved from the optimal solutions to the other subproblems appearing earlier in the sequence;
2. From the solution to one of the subproblems of the sequence (typically – but not always – the last one), we can deduce the optimal solution to the original problem.

Unlike other techniques such as branch-and-bound, dynamic programming cannot be used to solve any integer programming problem (try, for instance, to devise a dynamic programming approach to maximum independent set). But when it can, it often leads to efficient algorithms.

2 Knapsack

As a first example, consider the Knapsack problem.

Given: items $1, \ldots, n$, where item i has weight $w_i > 0$ and profit $p_i > 0$, and a capacity $\beta > 0$.
Find: Among sets $S \subseteq \{1, \ldots, n\}$ of items such that $\sum_{i \in S} w_i \leq \beta$ (that is, the total weight of items in S is at most β), one that maximizes $\sum_{i \in S} p_i$ (that is, one that maximizes the sum of the profits of the selected items).

Knapsack and its generalizations have many applications, for instance in portfolio optimization and in cryptosystems.

The following is an example of an integer programming formulation knapsack problem ($n = 4$; profits are $8, 11, 6, 4$; weights are $5, 7, 4, 3$ and $\beta = 14$).

$$\begin{array}{rl} \text{Max} & 8x_1 + 11x_2 + 6x_3 + 4x_4 \\ & 5x_1 + 7x_2 + 4x_3 + 3x_4 \leq 14 \\ & x_1, x_2, x_3, x_4 \in \{0, 1\} \end{array}$$

For $i \in \{1, \ldots, n\}$ and $j \in \{1, \ldots, \beta\}$, define $f(i, j)$ to be the optimal value of a knapsack solution that:

- uses items from $1, 2, \ldots i$ (possibly a subset of them);
- has total weight at most j.

As an example let us compute, for the instance above, $f(3, 10)$. We can do so by enumerating all solutions with items from $\{1, 2, 3\}$:

- $\emptyset$ is feasible and has total profit 0;
- $\{1\}$ is feasible and has total profit 8;
- $\{2\}$ is feasible and has total profit 11;
- $\{3\}$ is feasible and has total profit 6;
- $\{1,2\}$ has total weight 12, which exceeds our limit of 10, and it is therefore infeasible;
- $\{1,3\}$ is feasible and has total profit 14;
- $\{2,3\}$ has total weight 11, which exceeds our limit of 10, and is therefore infeasible;
- $\{1,2,3\}$ contains $\{1,2\}$ which is infeasible, and is therefore infeasible.

Thus, $f(3,10) = 14$.

Clearly, this enumerative approach is computationally too expensive for larger numbers of items and weight. Let us then compute $f(3,10)$ in a different manner. Let $S^* \subseteq \{1,2,3\}$ achieve $f(3,10)$. We distinguish two cases:

1. Either $3 \in S^*$. Thus, we know that item 3 will "consume" 4 units of capacity from our available 10 units of capacity. We can then fill the remaining 6 units of capacity in the optimal manner, using the first two items. By definition, this is exactly $f(2,6)$. Thus, in this case,
$$f(3,10) = \underbrace{f(2,6)}_{\text{optimal way to fill up to 6 units of capacity using items } 1,2} + p_3.$$
2. Or $3 \notin S^*$. In this case, the optimal solution coincides with the optimal solution to $f(2,10)$, since once we decide not to include item 3, its presence in the problem is immaterial. Thus we can set $f(3,10) = f(2,10)$.

Since we do not know which of the two is the largest, we pick the maximum. We have therefore:

$$f(3,10) = \max\{f(2,6) + p_3, f(2,10)\}.$$

In general, we have

$$f(i,j) = \max\{f(i-1, j-w_i) + p_i, f(i-1,j)\} \tag{1}$$

where we assume the boundary conditions $f(i,0) = 0$ (that is, the only solution of weight at most 0 is the empty solution) and $f(i,j) = -\infty$ for $j < 0$ and all i (that is, there is no solution of negative weight).

Let us know check that the recursive formula (1) verifies the two properties from Section 1.

1. *Every problem in the sequence can be easily solved from the optimal solutions to other problems appearing previously in the sequence*: true, as long as we compute $f(i,j)$ for smaller values of i first, since each $f(i,j)$ only depends on values $f(i-1,*)$.
2. *From the solution to one of the subproblems of the sequence (typically, the last one), we can deduce the optimal solution to the original problem*: true, since the value of the original knapsack problem is given by $f(n,\beta)$ (in the example above, $f(4,14)$).

Using the recursion (1), we can compute $f(i,j)$ by filling the table of all possible values. Rows denote values of i and columns values of j. We start from row 0 and fill a row completely before moving to the next. As an example, we highlight the two entries used to compute $f(3,10)$. More generally, each entry $f(i,j)$ is computed by using entry

$f(i-1,j)$ (that lies *vertically* immediately above) and entry $f(i-1,j-w_i)$ that lies *diagonally* above: one row up, w_i positions to the left).

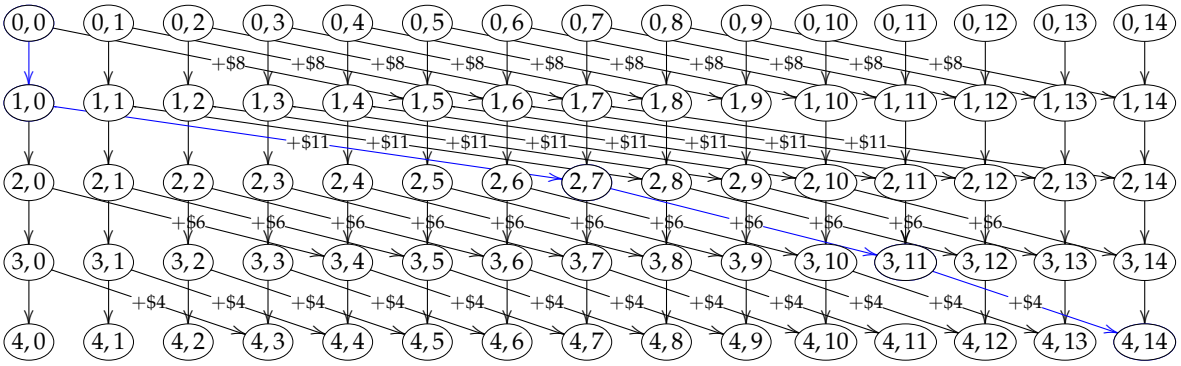
$f_i(j)$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	8	8	8	8	8	8	8	8	8	8
2	0	0	0	0	0	8	8	11	11	11	11	11	19	19	19
3	0	0	0	0	6	8	8	11	11	14	14	17	19	19	19
4	0	0	0	4	6	8	8	11	12	14	15	17	19	19	21

+$6

From the table above, we deduce that the optimal solution S^* to our original knapsack problem has value 21. However, we do not know yet which items are in S^*. We can reconstruct them by observing that for the computation of $f(4,14)$ (the last entry in the table), we used the formula

$$f(4,14) = \max\{f(3,11)+4, f(3,14)\}.$$

Since $f(3,11)+4 = 21 > 19$, we have $f(4,14) = f(3,11)+4$. Remembering how the recursive formula (1) was defined, we deduce $4 \in S^*$. We now move to $f(3,11)$ and iterate this argument, obtaining the path in blue, from which we deduce $S^* = \{2,3,4\}$ (each time we take a diagonal arc, we add an item to S^*; each time we take a vertical arc, we don't).



3 Dynamic programming: a general formulation

We can now abstract out the properties used to solve the knapsack problem and define more precisely the dynamic programming approach to (integer programming) optimization problems.

Associated to every subproblem, we define a function f that:

- takes in input two parameters: a *stage* i and a *state* j ; thus, the function is of the form $f(i,j)$;
- f is defined recursively; however, the value $f(i,j)$ can only depend on values $f(i',j')$ for $i' < i$.

In the knapsack problem above, stages are $1,\dots,n$ and states are $1,\dots\beta$.

4 Another investment problem

The next problem is similar to knapsack; however, each item can be selected multiple times, and leads to a non-linear profit. We motivate it with an application in portfolio optimization. There are three investments:

Investment 1: investing $x_1 > 0$ dollars yields $c_1(x_1) = 7x_1 + 2$ dollars, where $c_1(0) = 0$.
Investment 2: investing $x_2 > 0$ dollars yields $c_2(x_2) = 3x_2 + 7$ dollars, where $c_2(0) = 0$.
Investment 3: investing $x_3 > 0$ dollars yields $c_3(x_3) = 4x_3 + 5$ dollars, where $c_3(0) = 0$.

(Note that if no money is invested, then there is no yield; the yield is non-linear.)

We can invest \$6 and each investment must be a multiple of \$1. Following the framework from Section 3, we identify:

- stages $i = 1, \ldots, n$: in stage i, we consider only investments $1, \ldots, i$.
- states $j = 1, \ldots, 6$: in state j, the available budget is $\$j$.
- function $f(i, j) =$ maximum yield that we can obtain by investing up to j dollars into investments #1,...,#i.

In order to define the recursive formula, let us consider again a special case: $f(3,4)$. We distinguish several options, based on how much we invest on investment 3:

- We do not invest in investment 3. Thus, $f(3,4) = f(2,4)$;
- We invest exactly 1\$ in investment 3. Thus, $f(3,4) = f(2,3) + c_3(1) = f(2,3) + 9$;
- We invest exactly 2\$ in investment 3. Thus, $f(3,4) = f(2,2) + c_3(2) = f(2,2) + 13$;
- We invest exactly 3\$ in investment 3. Thus, $f(3,4) = f(2,1) + c_3(3) = f(2,1) + 17$;
- We invest exactly 4\$ in investment 3. Thus, $f(3,4) = f(2,0) + c_3(4) = f(2,0) + 21$.

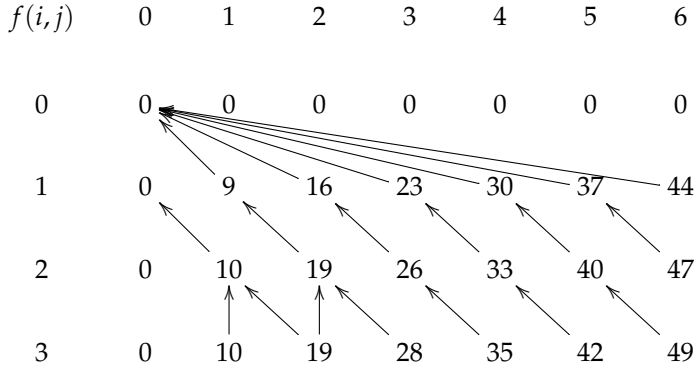
Similarly to the knapsack case, we do not know which option is correct, thus we pick the maximum:

$$f(3,4) = \max\{f(2,4), f(2,3) + c_3(1), f(2,2) + c_3(2), f(2,1) + c_3(3), f(2,0) + c_3(4)\} = \max_{k=0,\ldots,4} \{f(i-1, j-k) + c_3(k)\}.$$

We can now write the generic recursive formula for the function f

$$f(i,j) = \begin{cases} 0 & i = 0 \\ \max\limits_{k \in \{0,1,\ldots,j\}} \left\{ f(i-1, j-k) + c_i(k) \right\} & i \geq 1 \end{cases} \tag{2}$$

We can compute $f(i,j)$ with a table similar to the one for knapsack, where again stages are represented by rows. For every entry (i,j) of the table, we draw an arrow from (i,j) to the entry $(i-1, j')$ that achieves the maximum in (2).



5 Scheduling

We have 7 jobs $j_1, j_2, \ldots, j_7$ that need to be completed. Any job can be scheduled on any of 2 available machines, but no two jobs can be scheduled on the same machine in overlapping times. The processing times $p_1, p_2, \ldots, p_7$ do not depend on the machine they are scheduled on, and are as follows:

job j_i	j_1	j_2	j_3	j_4	j_5	j_6	j_7
processing time p_i	10	8	6	5	4	4	3

The goal is to schedule jobs on machines as to minimize the time when all jobs have been completed ("completion time"). For instance, if we schedule jobs j_1, j_2, j_4 on machine 1 and the remaining jobs j_3, j_5, j_6, j_7 on machine 2, the completion time is 23, which is the maximum of the time the first machine has executed all its jobs – 23 – and the time the second machine has executed all its jobs – 17. Note that the order in which jobs are executed does not matter; it only matters on which machine they are executed.

We can formulate the problem as a dynamic program as follows:

- stages: at stage i we schedule first i jobs $j_1, \ldots, j_i$;
- states: a state is a pair (t_1, t_2) denoting that we used up to time t_1 on machine #1 and time t_2 on machine #2;
- function: $f(i; t_1, t_2) = \begin{cases} 1 & \text{if it is possible to schedule first } i \text{ jobs on the two machines using } t_1 \text{ time} \\ & \text{on machine \#1 and } t_2 \text{ time on machine \#2;} \\ 0 & \text{otherwise.} \end{cases}$

Let us again start with an example. Suppose we want to compute $f(4; 15, 18)$. Then we have the following options:

- We schedule j_4 on machine 1. Thus, $f(4; 15, 18) = 1$ if and only if $f(3; 10, 18) = 1$ (where $10 = 15 - p_4 = 15 - 5$).
- We schedule j_4 on machine 2. Thus, $f(4; 15, 18) = 1$ if and only if $f(3; 15, 13) = 1$ (where $13 = 18 - p_4 = 18 - 5$).

Thus, $f(4; 15, 18) = 1$ if and only if at least one of $f(3; 15, 13)$ and $f(3; 10, 18)$ is equal to 1; and $f(4; 15, 18) = 0$ if $f(3; 15, 13) = f(3; 10, 18) = 0$. Equivalently,

$$f(4; 15, 18) = \max\{f(3; 15, 13), f(3; 10, 18)\}.$$

In general, to compute $f(i; t_1, t_2)$, we either execute job j_i on machine #1 or on machine #2, and we pick the option that is achievable (if any). In formulas,

$$f(i, t_1, t_2) = \begin{cases} 0 & t_1 < 0 \text{ or } t_2 < 0 \\ 1 & i = 0 \\ \max\left\{f(i-1; t_1 - p_i, t_2),\ f(i-1; t_1, t_2 - p_i)\right\} & i \geq 1 \end{cases}$$

While the stages clearly go from 1 to the number of jobs, the number of states is less clear. A safe choice is to let both t_1 and t_2 go from 1 to $\sum_{i=1}^{n} p_i$, since the completion time cannot exceed $\sum_{i=1}^{n} p_i$.

As for the optimal solution: we want to find the smallest t^* such that $f(n; t^*, t^*) = 1$ (with n being the number of jobs). Indeed, when this happens, there is a solution whose completion time is t^*, while there is no solution whose completion time is $t < t^*$, since for such t we would also have $f(n; t, t) = 1$.

You can verify that, in the example above, the optimal solution is given by scheduling on the first machine jobs j_2, j_4, j_6, j_7 and on the second machine jobs j_1, j_3, j_5.